Web Application Requirements Specification and Analysis App

Sprint Report

DEV-OPS
CHARALAMPOS KOUNNAPIS AM:5401,
PANAGIOTIS CHRISTODOULOU AM:5501

VERSIONS HISTORY

Date	Version	Description	Author
14/05/2026	1.0	It's the final version of the requirements specifications and analysis app (web application)	Charalampos Kounnapis Panagiotis Christodoulou

1 Introduction

1.1 Purpose

This document presents the **final** sprint report of the Web Application Requirements Specification and Analysis App.

1.2 Document Structure

The rest of this document is structured as follows. Section 2 describes our Scrum team and specifies this Sprint's backlog. Section 3 specifies the detailed Use Cases. Section 4 presents the architecture, class diagrams, and CRC cards of the application. Section 5 presents the testing

1.3 Coding Environment

The Project was created and tested using Eclipse IDE (In order to execute the project you must have Spring Tools from Help →Eclipse Marketplace). To run the program first open Eclipse then click on File → Import →Maven → Existing Maven Projects then on the pop-up window click Browse and select the Project folder and click finish. To run navigate to src/main/java click and expand com.reqapp package and right click RequirementsAppApplication.java → Run As → Spring Boot App.

2 Scrum team and Sprint Backlog

2.1 Scrum team

Product Owner	CHARALAMPOS KOUNNAPIS
Scrum Master	PANAGIOTIS CHRISTODOULOU
Development Team	CHARALAMPOS KOUNNAPIS PANAGIOTIS CHRISTODOULOU

2.2 Sprints

List below the sprints that you performed and the user stories that have been realized in each Sprint

Sprint No	Begin Date	End Date	Number of weeks	User stories
1	16/03/26	30/03/26	2 weeks	US1 – Account Creation and Log in US2 – Developers Profile US3 – Log out
2	30/03/26	06/04/26	1 week	US4 – List of all projects US5 – Creation of a project and the organization of USC and CRC Cards US6 – Delete a Project
3	06/04/26	19/04/26	2 Weeks	US7 – Create a Use Case and link Actors US8 – Edit a Use Case US9 – view List of Use Case US10 – Delete a Use Case
4	20/04/26	27/04/26	1 Week	US11 – Create a CRC Card US12 – Edit a CRC Card US13 – Link CRC Card to Use Case US14 – Delete CRC Card
5	28/04/26	13/05/26	2,5 weeks	US15 – Generate Use Case Diagram Script (PlantUml and Nomnoml) US16 – Generate Class Diagram Script(PlantUml and Nomnoml) US18 – Project sharing (add team mates to Project) US19 – Comment on UseCases and CRC cards Creation of Help page (User guidelines and main functionalities of application)

3 Use Cases

Specify the concrete Use Cases that describe the interaction of the user with the applications, as derived from the abstract user stories. Give a **UML Use Case diagram** and the **detailed use case descriptions**.

3.1 Use Case 1 – User Registration and Login

Use case ID	UC1
Actors	User
Pre conditions	1. The user is not logged in 2. The system is running
Main flow of events	1. The use case starts when the user opens the login page 2. The user enters username and password 3. The user submits the login form 4. The system validates the credentials 5. The system authenticates the user 6. The user is redirected to the dashboard
Alternative flow 1	1. If the user's credentials are invalid the system displays an error message 2. If the user submits empty fields the system displays an error message
Alternative flow 2	1. If the user doesn't have an account 2. The user presses the register button and the user redirects to the register page 3. The user enters their personal details (first/last name, email, username, password) 4. The system checks for duplicate username 5. The system saves the new account 6. The system redirects the user to the login page
Alternative flow 3	1. The user retries to login
Post conditions	1. The user is authenticated and logged in 2. A session is created

3.2 Use Case 2 – Manage Projects

Use case ID	UC2
Actors	User
Pre conditions	1. The user is logged in
Main flow of events	1. The user views “My projects” page 2. The user enters project name and a description 3. The user clicks the “Create” button 4. The system validates project name input 5. The system stores the project in the database and refreshes the project list
Alternative flow 1	1. If project name is empty the system shows validation error 2. If the user enters an existing project name the system shows validation error
Alternative flow 2	1. The user clicks the “Delete” button in order to delete a project from the project list 2. The system ask for confirmation, then deletes the project
Post conditions	1. If a project is created, it is stored in the system and shown in the project list 2. if a project is deleted, it is removed from the system

3.3 Use Case 3 – Manage Use Cases

Use case ID	UC3
Actors	User
Pre conditions	1. The user is logged in 2. A project exists 3. The user has access to the selected project (owner or team mate)
Main flow of events	1. The user clicks the "Use Cases " button on the existing project 2. The user is transferred to the use case page and clicks "Add New Use Case" button 3. The user fills in title, actors, preconditions, main flow, postconditions 4. The user links any existing CRC cards by selecting it 5. The user submits the form 6. The system saves and refreshes the Use Case page
Alternative flow 1	1. The user can edit or delete an existing Use Case by clicking the given buttons
Alternative flow 2	1. The user views the list of Use Cases in the selected project 2. The user writes a comment in the comment box in the specific use case 3. The user clicks the " Post " button 4. The system saves the comment with the User and adds it to the Use Case 5. The system displays the comment
Post conditions	1. The use case is stored in the project

3.4 Use Case 4 – Manage CRC Card

Use case ID	UC4
Actors	User
Pre conditions	1. The user is logged in 2. A project exists 3. The user has access to the selected project (owner or team mate)
Main flow of events	1. The user clicks the "CRC Cards" button on the existing project 2. The user is transferred to the crc card page and clicks "Add New CRC Card" button 3. The user fills in Class Name, Responsibilities and Collaborations 4. The user submits the form 5. The system saves and refreshes the CRC Card page
Alternative flow 1	1. The user can edit or delete an existing CRC card by clicking the given buttons
Alternative flow 2	1. The user views the list of CRC card in the selected project 2. The user writes a comment in the comment box in the specific CRC card 3. The user clicks the " Post " button 4. The system saves the comment with the User and adds it to the CRC card 5. The system displays the comment
Post conditions	1. The CRC card is stored in the project

3.5 Use Case 5 – Generate UML Diagram

Use case ID	UC5
Actors	User
Pre conditions	1. The project contains at least one Use Case or CRC card 2. The user is logged in. 3. The user has access to the selected project.
Main flow of events	1. The user selects "Generate Diagrams" button in a existing project 2. The user selects the type of diagram (Use Case diagram or Class diagram) 3. The user selects the tool format (PlantUML or Nomnoml) 4. The user clicks the "Generate" button 5. The system generates the selected script 6. The system displays the script
Alternative flow 1	There are no alternative flows for this use case
Post conditions	1. The generated script can be copied

3.6 Use Case 6 – Share Project

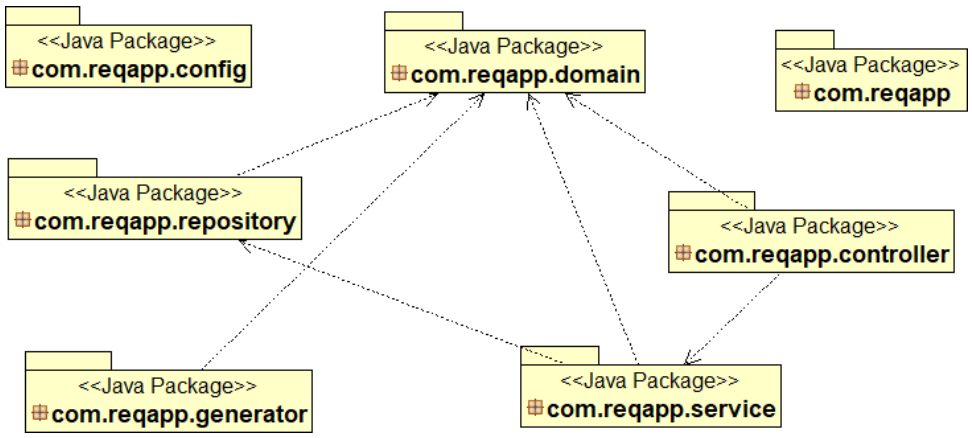
Use case ID	UC6
Actors	User (Owner) Teammate
Pre conditions	1. The user is the owner of the project
Main flow of events	1. The user is on the “My Projects” dashboard 2. The owner enters the teammates username in the “add teammate” field 3. The owner clicks the “Add” button 4. The system adds the teammate to the project
Alternative flow 1	1. If the username doesn’t exist no teammate is added
Post conditions	1. The teammate can see and edit the shared project

4 Design

4.1 Architecture

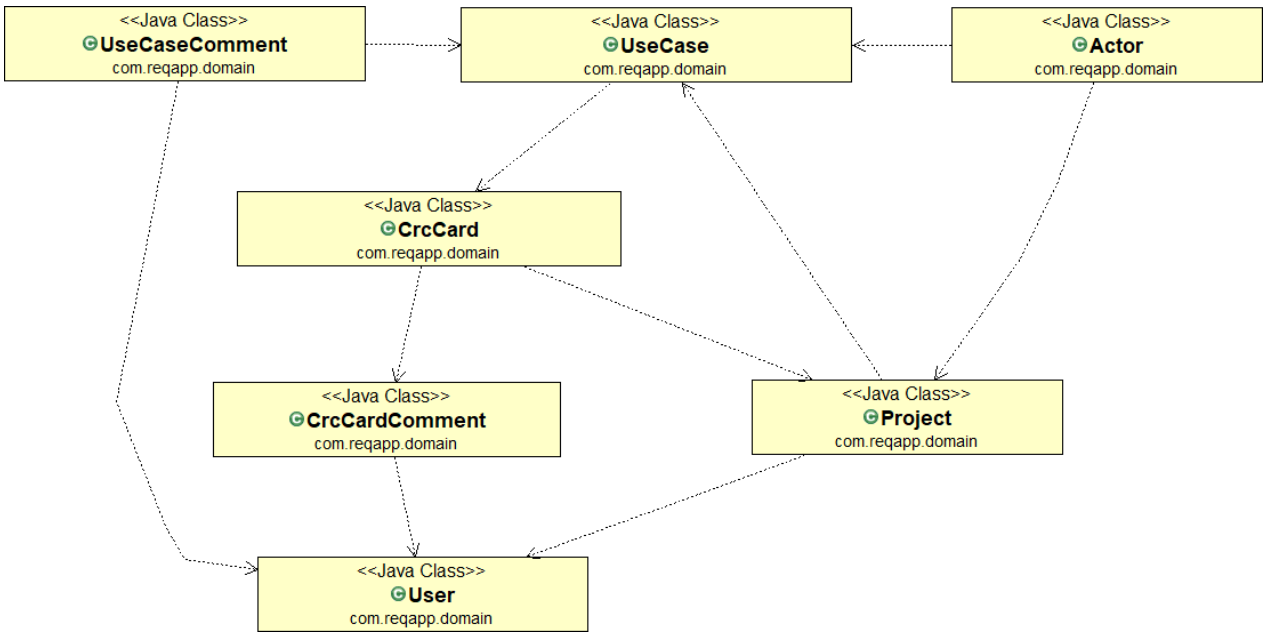
The system follows a layered architecture consisting of controller, service, repository, domain layers and generator packages. The controller layer handles the HTTP requests and returns Thymeleaf views. The service layer contains the main logic of the application. The repository layer is responsible for database access. The generator package contains the classes responsible for generating PlantUML and Nomnoml scripts.

The following diagram shows the architecture of the system.



4.2 Design

The following UML class diagram shows the structure of the system, including the main domain classes and their relationships.



4.3 The following CRC cards describe the responsibilities and collaborations of the main classes in the system.

Class Name: User	
Responsibilities: ▪ Manage user account ▪ Store user information (first/last name, username, password, email) ▪ Own and manage projects ▪ Create comments	**Collaborations:** ▪ Project

Class Name: Project	
Responsibilities: ▪ Store project information (name, description) ▪ Manage use cases (UC) ▪ Manage CRC cards ▪ Manage shared users ▪ Stores Project owner	**Collaborations:** ▪ User ▪ UseCase ▪ CrcCard ▪ Actor

Class Name: UseCase	
Responsibilities: ▪ Store use case data (title, precoditons, main flow, post conditions) ▪ Manage actors ▪ Manage comments ▪ Links with CRC cards	**Collaborations:** ▪ Project ▪ Actor ▪ CrcCard ▪ UseCaseComment

Class Name: Actor	
Responsibilities: ▪ Represents an actor in a use case ▪ Store actor name ▪ Belongs to a project ▪ Linked to one or more use cases	**Collaborations:** ▪ UseCase

Class Name: CrcCard	
Responsibilities: ▪ Store Crc Card data (class name, responsibilities, collaborations) ▪ Belongs to a project ▪ Manage comments	**Collaborations:** ▪ Project ▪ CrcCardComment

Class Name: UseCaseComment	
Responsibilities: ▪ Store comment text ▪ Link to a user (author) ▪ Link to a use case (UC)	**Collaborations:** ▪ User ▪ UseCase

Class Name: CrcCardComment	
Responsibilities: ▪ Store comment text ▪ Link to a user (author) ▪ Link to a CRC card	**Collaborations:** ▪ User ▪ CrcCard

5 Testing

The application was tested using JUnit, Mockito, Spring Boot testing facilities and the H2 database in order to validate the correct implementation of the user stories and the interaction between the different layers of the system.

5.1 Unit Tests

Unit tests were developed for the service layer and the diagram generation package.
Mockito was used to mock repository dependencies when needed.

- Test class: **DiagramGeneratorTest**
- Description: Tests the diagram generation logic for Use Case diagrams and Class diagrams.

Tested functionality:

- Checks that GeneratorFactory creates the correct generator for PlantUML or Nomnoml.
- Checks that Use Case Diagram scripts contain the expected actors and use cases.
- Checks that Class Diagram scripts contain the expected CRC card information.
- Verifies the basic behavior.

- Test class: : **ProjectServiceTest**
- Description: Tests project management and project sharing logic.

Tested functionality:

- Checks that projects can be saved.
- Checks that projects can be deleted.
- Checks that an existing user can be added as a teammate.
- Checks that the project owner has access to the project.
- Checks that a teammate has access to the shared project.
- Checks that an unrelated user does not have access.

- Test class: **ProjectServiceCommentTest**
- Description: Tests the simple commenting functionality.

Tested functionality:

- Checks that a comment can be saved for a Use Case.
- Checks that a comment can be saved for a CRC Card.
- Checks that each comment stores the correct text and author.
- Checks that each comment is linked to the correct Use Case or CRC Card.

5.2 Repository Tests

- Test class: **ProjectRepositoryTest**
- Description: Tests the custom Spring Data JPA query using the H2 database.

Tested functionality:

- Checks that a project owner can retrieve their own project.
- Checks that a teammate can retrieve a project shared with them.
- Checks that an unrelated user cannot retrieve the project through the owned/shared project list.

5.3 Integration Tests

Integration tests were performed to validate the interaction between controller, services, repositories and the database layer.

- Test class: **SimpleSecurityAndAccessIntegrationTest**
- Description: Tests basic Spring Security behavior using Spring Boot test support.

Tested functionality:

- Checks that an unauthorized user who tries to access the protected projects page is redirected to the login page.

5.4 Acceptance Tests

Acceptance tests were performed to ensure that the implemented user stories satisfy the functional requirements of the application.

The tests validated:

- US1 – US3: User registration, login, logout, and profile management.
- US4 – US6: Project management operations (creation, listing, deletion)
- US7 – US10: Use case management operations (creation, editing, listing, deletion and linking to a CRC card)
- US11 – US14: CRC card management operations (creation, editing, deletion)
- US15 – US 16: Generation of PlantUML or Nomnoml diagram scripts
- US18: Sharing a project with teammate.
- US19: Adding comments to Use Cases and CRC Cards.
- NF2(Usability): Accessing the Help/User Guidelines page.

All automated test were executed successfully: